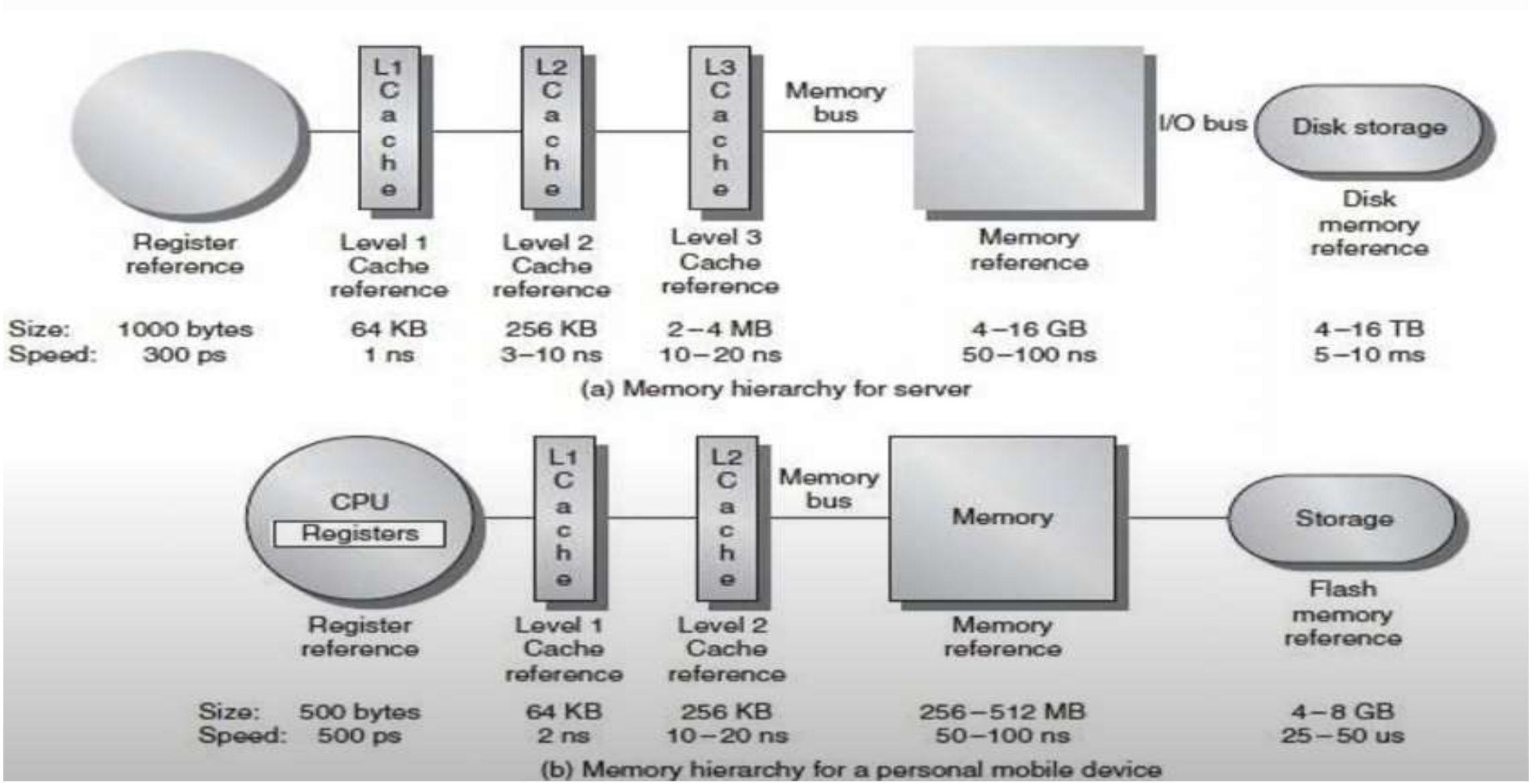
Advanced Computer Architecture CMP 511

Memory Technology

Department of Computer Engineering

Pokhara University

Level in the hierarchy



Locality in memory hierarchy

- Principle of locality states the access of a relatively small portion of address pace at any instant of time
- Temporal and spatial in programs includes loops, arrays, instructions execution in sequential manner
  - Temporal – same item may be needed
  - Spatial- nearby items may be needed in future
- Memory hierarchy
  - Multiple levels with the different speed and size
  - Faster memory implies more expensive bit, as a result we use small size
- So it is desirable to have as much memory as available in the cheapest technology while providing access at the speed offered by the fastest memory

Memory hierarchy

- Every upper level is the subset of the lower level
- Essentially all the data is stored in lower level
- Level closer to the processor (upper level) is the subset of any level further away
- So memory hierarchy makes sure that the data are subset of each other and data gets transferred between two adjacent levels
- Multiple levels implies data will be copied from any two adjacent levels at a time.
- Minimum unit of information that is present is called a block or line

Memory hierarchy

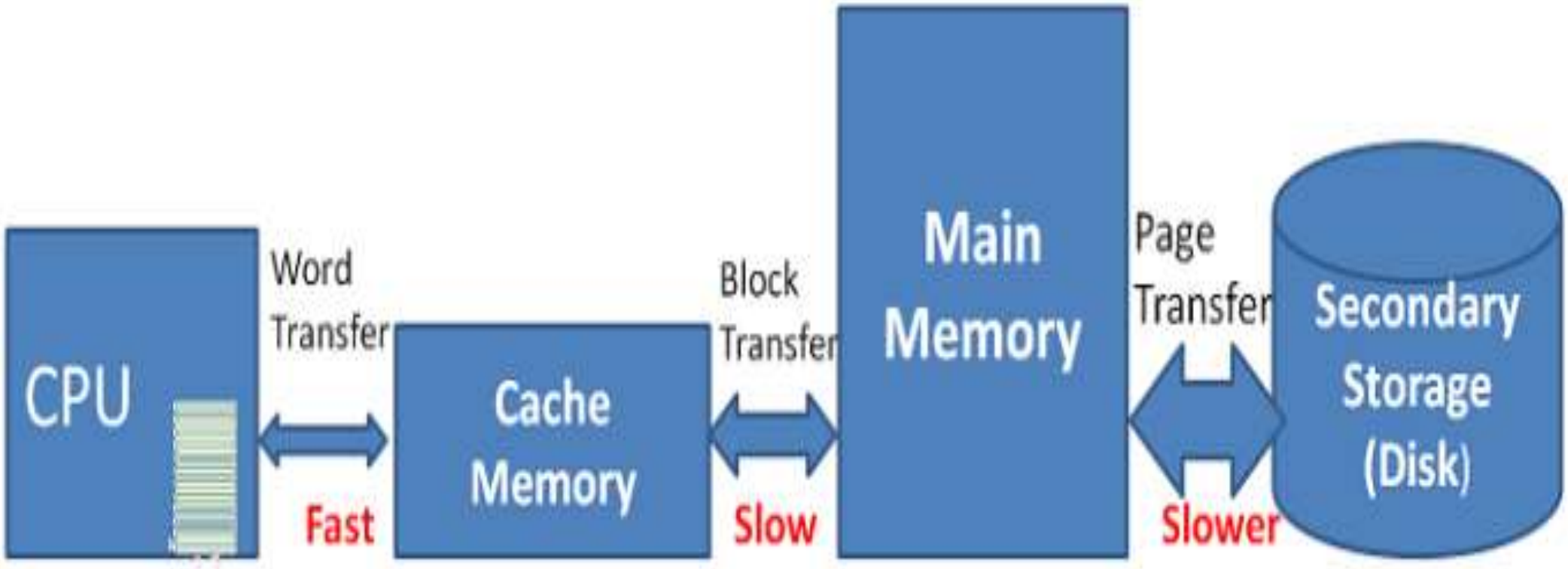
- The way the memory hierarchy is made will directly affect the performance of the system
- Memory is critical to performance since most of the programs spends most of the their time accessing memory.
- Hereby it makes memory system to be a major factor for determining performance.
- Computer designers need to pay attention on how to improve the performance of the memory system, if not taken care upon then its will affect the other applications running on the system
  - Depending on the memory performance, the operating system is going to manage the memory and the input-output
  - and the compiler is also going to generate the code depending upon how good is the memory performance

  Hence it is a mandatory to make sophisticated mechanism to improve the performance of the memory system

Memory hierarchy - metrices

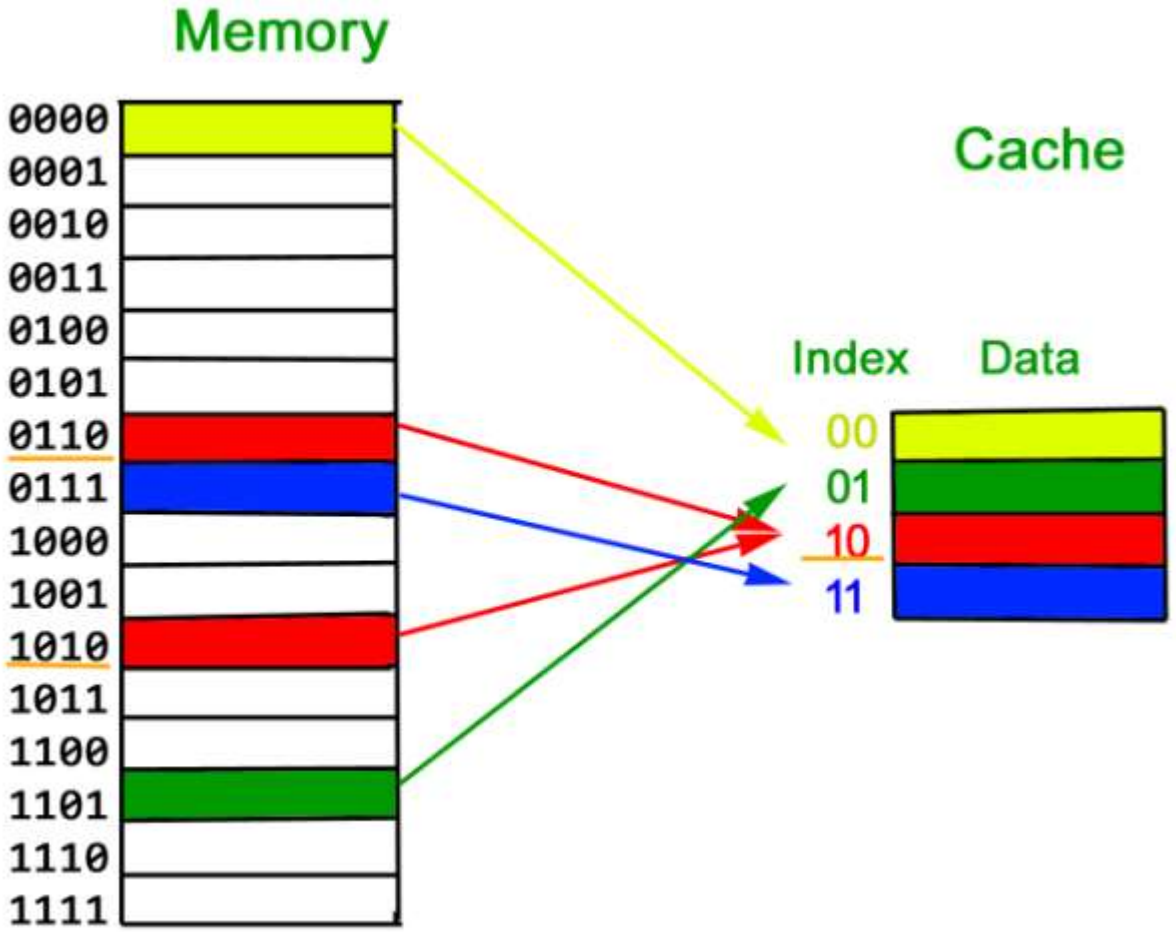
- **Hit** ➔it is the requested data found in the upper layer
- **Miss** ➔otherwise
- **Hit rate➔ hit ratio** = fraction of memory accesses found in the upper layer
- **Miss rate** ➔ 1 – hit rate
- **Hit time** ➔ it is the time to access the upper layer
- **Miss-penalty** ➔ it is a time to replace a block in the upper layer

Cache Basics

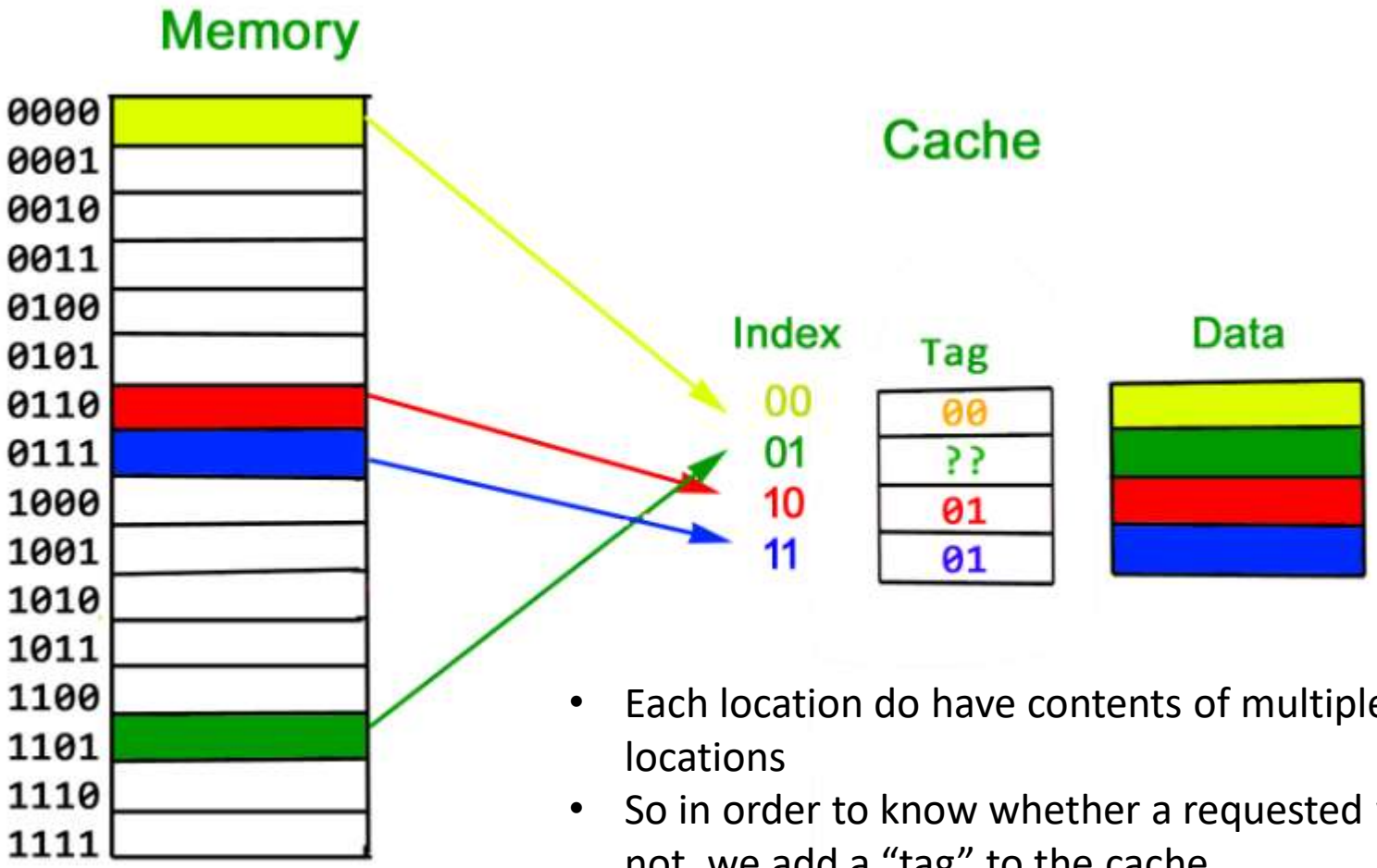


Cache Design

- Block number = block address MOD (Number of blocks in cache)



Cache Design



- Each location do have contents of multiple different memory locations
- So in order to know whether a requested word is in the cache or not, we add a “tag” to the cache
- By definition, index field of any address must be that block number, and generally in the “tag” architects omit the index field as it is redundant

Cache Design

- Generally when we start the system, the cache may have garbage value preloaded
- Hence to identify the garbage value from the real value, there is a tendency of adding another bit to the cache called the “valid bit”
- So when we start the system, we do reset all these valid bits so that no accidental hits can happen inside the cache
- All the valid bits are made to zero and when the program loads the actual value, the valid bits is turned to 1

Cache Design

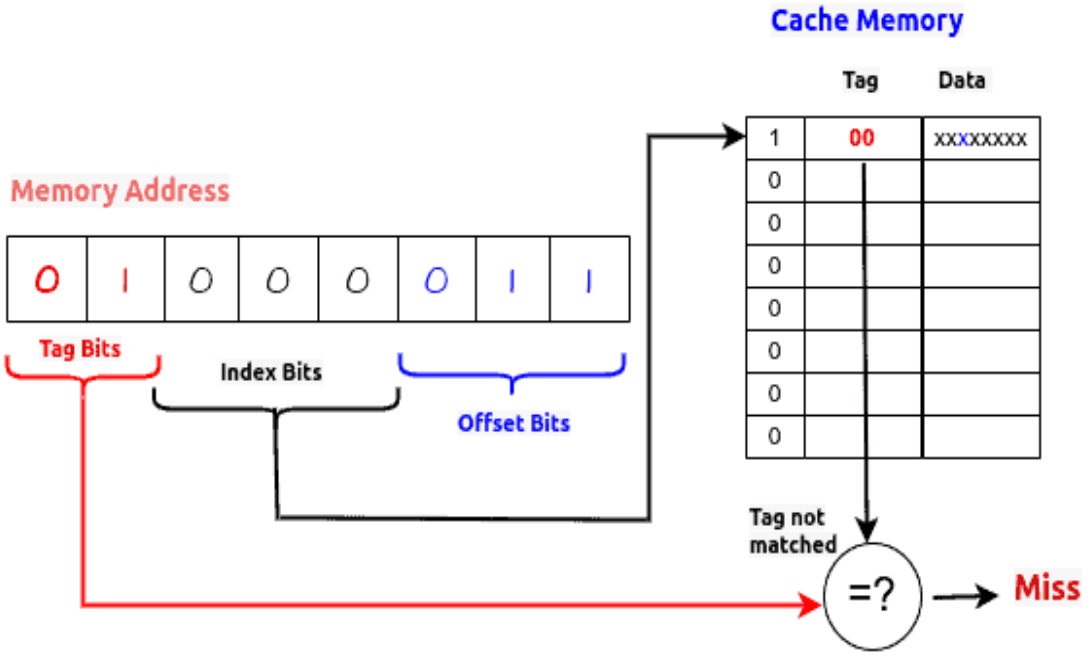
Let us consider a case where

Main memory size = 4GB

Cache size = 1MB

Block size = 4KB

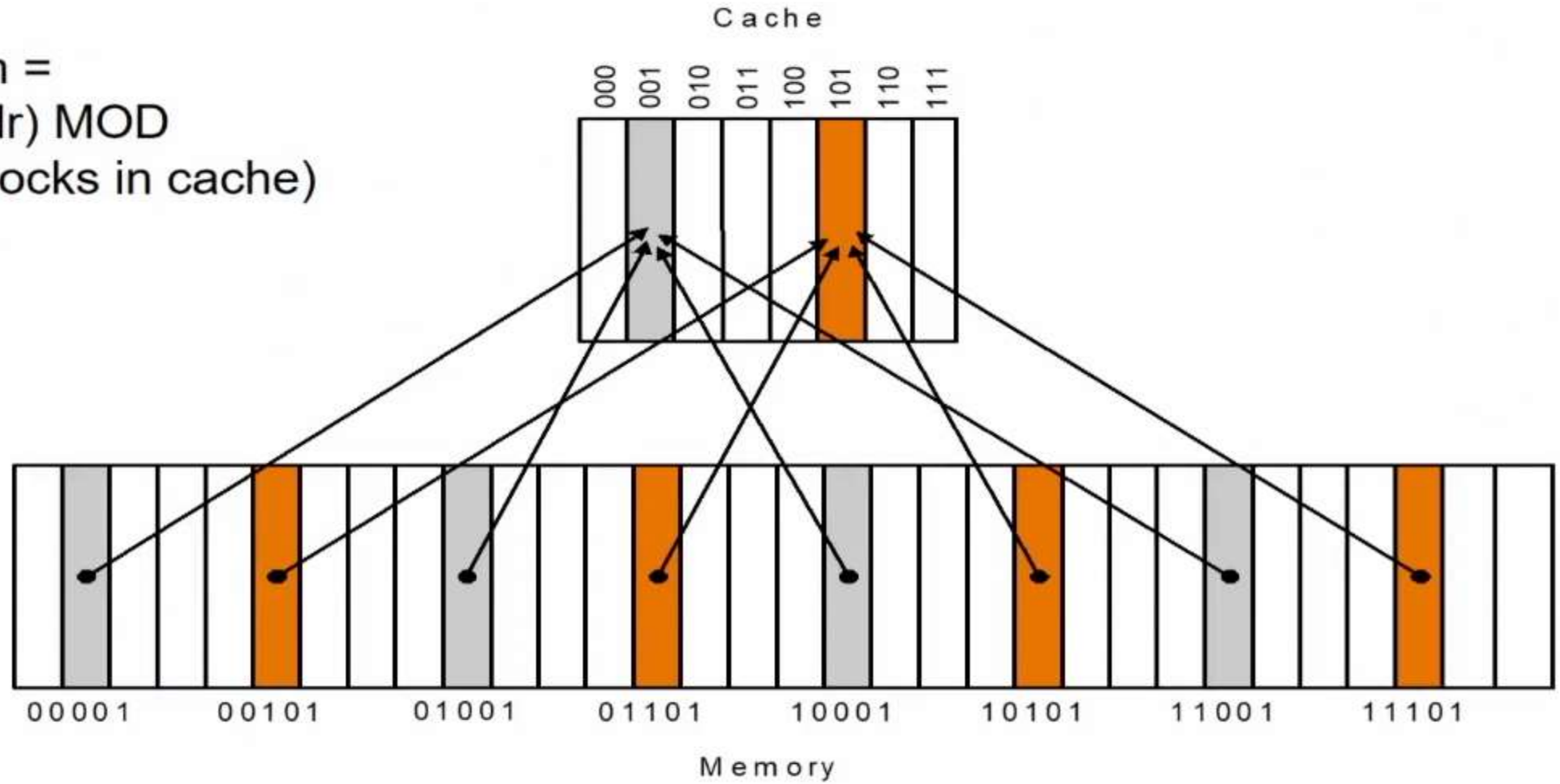
Cache Design



- Let’s consider an 8-bit memory address 01000011 and cache with 8 lines, each containing 8 blocks. We use three bits for the index and three for the offset (since cache lines hold 8 = 2^3 blocks). The remaining two bits are for the tag.
- The cache controller retrieves the tag associated with the cache line at the first index because that’s where the index bits 000 point. Then, it compares it to the tag derived from the memory address and present in the cache, which is 01 in this example. The tags don’t match, which means the requested memory block is not in the cache, so we have a cache miss.
- Now, the cache controller fetches the requested data from the main memory address = 01000011 and stores it in the cache line corresponding to the first index. If there is already data present in that cache line, it will be replaced with the new block.
- If the tags matched, the controller would retrieve the third word from the first cache line because that’s what the offset bits (011) specify.

Direct mapped

Block num =
(block addr) MOD
(num of blocks in cache)



Pentium Cache

- It is an improvement of directing mapping
- Set associative memory
- Here each word of cache can store two or more words of memory under same index address
- Each data word stored together with it is tag and no of tag-data item in one word of cache is said to form a set

Cache Design Strategy: Block Placement

- Keep it in one location
  - Direct mapping arrangement
- Keep it anywhere or
  - Fully associative arrangement
- Give some limited choice
  - Set associative arrangement
  - Block number = block address MOD (Number of sets in cache)

Cache Design Strategy: Block Placement

- Suppose cache has 8 block; now we have to find where does the address 11 goes in the cache for the given arrangement as under
  - direct mapping
  - Fully associative mapping
  - Two way set associative mapping

Cache Design Strategy: Block Identification

- Our case design strategy primarily involves identifying a block in the upper level
- We have divided the address into a tag, and index and byte offset
- So index and byte offset are taken care of to reach a particular location of the cache and whether to read or not is identified by the tag portion.
- As the associativity increases blocks per set increases, index size decreases, tag size increases

Cache Design Strategy: Block Replacement

- Cache = small in size
- Memory = big in size
- So it a prime concern to know which block should be replace in a cache miss?

Cache Design Strategy: Block Replacement

- Direct mapping is a simple hardware arrangement where every block has got just one choice since everything is pre-decided
- In Fully associative (and in set associative) cache there are many choices. Here you can decide who goes out?
  - Random
  - LRU (Least Recently used)
  - FIFO (First-in first-out)
  - Optimal algorithm

Cache Design Strategy: Block Replacement

- Random
  - Here we will use a pseudo random generator to decide which block goes out and
  - The seed will be fixed so that we can reproduce the things for debugging
- LRU
  - Replaces the block (oldest) that has not been used for the longest time
  - It emphasis of the locality of reference concept ➔ since the block is not used right now, it is also likely that it will not be used in the future
  - However the hardware complexity increases since we need to keep track about which block was used when and etc. Hereby the hardware complexity of this decision is tremendously high

Cache Design Strategy: Block Replacement

- FIFO
  - It decides the oldest block in time. So the block which was accessed much earlier will definitely be removed
  - It is the approximated version of LRU and removes the oldest block rather than the LRU
- Optimal algorithm
  - Replaces the block that will not be used for the longest time (must know the future)
  - Is supposed to be the best algorithm

Cache Design Strategy: Write strategy

- Consider a example of MIPS (Microprocessor without Interlocked Pipelined Stages and we have a program where store = 20% and load = 36%
- What is the memory traffic on a write?
  - 20%/(20%+36%+100%) = 12.8%
- What is the traffic going to the data cache?
  - 20%/(20%+36%) = 35.7%

Cache Design Strategy: Write strategy

- So it is a prime concern to improve the writing ability
- Basically we have two write options to make write faster
  - Write through – any change in the first level is percolated to the second level
  - Write back – here we don’t go all the way to the main memory rather we keep on accumulating changes in the small cache and only send them to the main memory

Virtual Memory

- The physical size of our memory is limited
- Since the program may or may not fit in the RAM, our concern is now how to get more memory?
- Here we give illusion of a large storage available to each program
- It means we take help of the secondary storage to logically expand RAM capacity
- By the concept of virtual memory, we can now run the program having greater capacity than that of the size of the main memory
  - This job is accomplished because some part of the secondary memory is taken by the processor as its physical memory

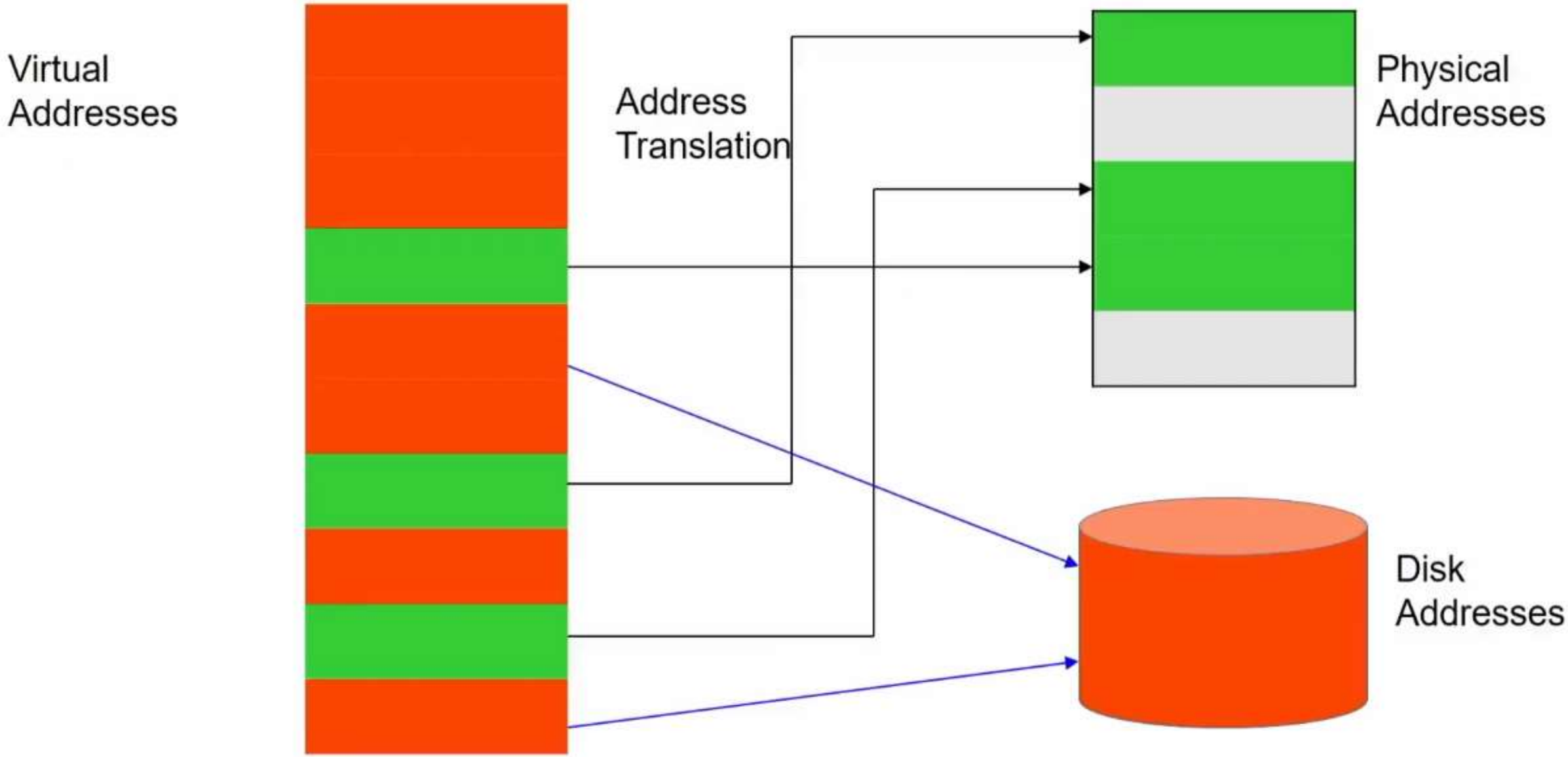
Virtual Memory

- Technically, now the main memory acts as a cache to the secondary storage
- Apart from large storage, virtual memory is advantageous by several reasons as under
  - Since RAM is shared – it allows safe sharing of the memory among multiple program. Every process is going to store its data into the RAM
    - **Is it possible that another process can read our data??**
  - ***Greatest motivation*** - It allows the single program to exceed the size of the RAM (your program size is greater than the size of the RAM)
  - Before the origination of virtual memory – for a limited capacity, programmers need to write a small programs or need to split the program into smaller programs or modules which will then be loaded turn by turn to be executed

Virtual Memory

- Each program has its own virtual address space
- Virtual memory implements the translation of a program address space to the physical address
- This translation process enforces the protection of a programs address from other virtual machines/processes
- Virtual address ← Physical address
  (processor) (RAM)

Virtual memory



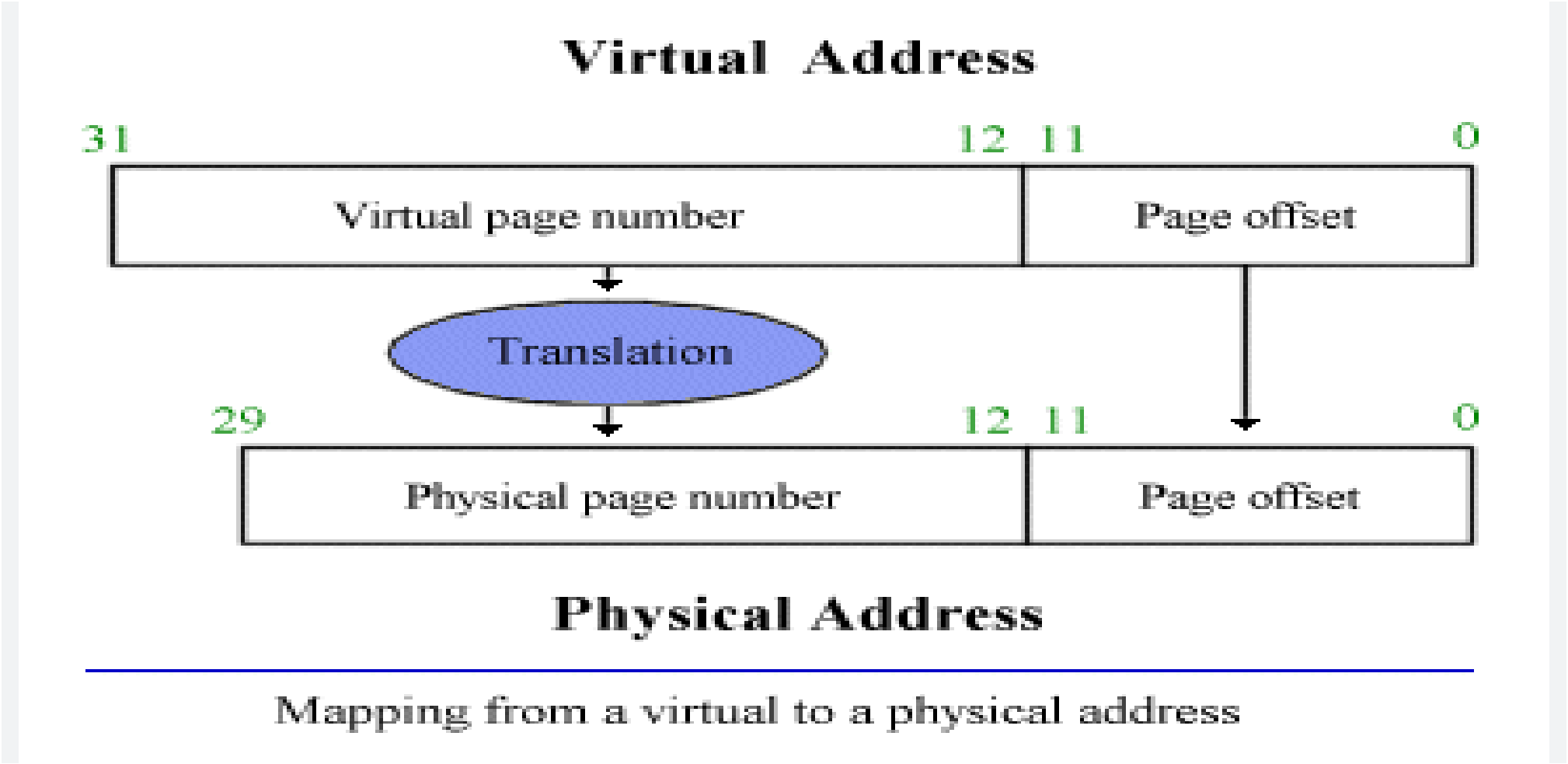
Some terminologies related to Virtual Memory

- The virtual memory block is called a **page**
- The virtual memory miss is called a **page-fault**
  - Page fault occurs when the page is not there in the RAM and that page needs to be brought from the main memory
  - It is to be noted that page-fault takes longer time to handle, therefore OS does it, meanwhile processor is scheduled for another process
  - When cache miss occurs, it is handled by the hardware
- If the pages of the virtual memory are taken to the main memory as per the demand of the processor, then we call it as **demand paging**

Address translation

- It is the work of processor to produce virtual address and this virtual address has to be translated to physical address by the use of both hardware and software
  - So the process of doing this translation is called **address mapping or address translation**
- Virtual address = page number + page offset
- And the number of bits in the page-offset tells us the size of the page
- The number of pages addressable by the virtual address need not necessarily match the number of pages addressable by the physical address
- So the small RAM is now accessible to many more virtual pages which gives us the illusion of having an essentially unbounded amount of memory

Can we run a program having greater size than that of the size of the memory?



Mapping from a virtual to a physical address

- **Hereby we can run a program which is 4 GB wide using only 1 GB size of RAM.**

Page Table

- Since the disk is far – obviously it is going to take more time to load ≈ millions of clock cycle latency
- Typical pages sizes are normally of 4 KB or 16 KB but modern system do have 64 KB pages
- Embedded system has tiny pages of ≈ 1 KB; however depending upon the system requirement the page size can be adjusted
- Whenever a page comes, it can be place anywhere in the RAM
  - RAM design has to be fully associative such that we can reduce the number of page fault. Hereby optimal algorithms can reduce page faults
- Write through policy is not used here
- Every process has got its own page table to ensure protection and quick access

Translation Lookaside Buffer (TLB)

- Page table is stored in the memory
- So each memory access needs twice the time
  - One memory access to get physical address (i.e. to say read page table)
  - Second memory access to get the data
- Key to improve performance is locality of reference to the page table
  - When a translation of a page table entry is used, it is likely to be used again in the near future. Because the references to the words in the page have both spatial and temporal locality
- Modern processors include a special cache to keep track of the recently used translations
- This special address translation cache is called the translation Lookaside Buffer (TLB), also called translation cache

Example 4.7 The design of a memory hierarchy

Consider the design of a three-level memory hierarchy with the following specifications for memory characteristics:

Memory level	Access time	Capacity	Cost/Kbyte
Cache	$t_1 = 25$ ns	$s_1 = 512$ Kbytes	$c_1 = \$0.12$
Main memory	t_2 = unknown	$s_2 = 32$ Mbytes	$c_2 = \$0.02$
Disk array	$t_3 = 4$ ms	s_3 = unknown	$c_3 = \$0.00002$

The design goal is to achieve an effective memory-access time t = 850 ns with a cache hit ratio h1 = 0.98 and a hit ratio h2 = 0.99 in main memory. Also, the total cost of the memory hierarchy is upper-bounded by \$1,500. The memory hierarchy cost is calculated as

$$C = c_1 s_1 + c_2 s_2 + c_3 s_3 \leq 1{,}500 \tag{4.7}$$

The maximum capacity of the disk is thus obtained as $s_3 = 40$ Gbytes without exceeding the budget.

Next, we want to choose the access time (t_2) of the RAM to build the main memory. The effective memory-access time is calculated as

$$t = h_1 t_1 + (1 - h_1) h_2 t_2 + (1 - h_1)(1 - h_2) h_3 t_3 \leq 850 \tag{4.8}$$

Substituting all known parameters, we have $850 \times 10^{-9} = 0.98 \times 25 \times 10^{-9} + 0.02 \times 0.99 \times t_2 + 0.02 \times 0.01 \times 1 \times 4 \times 10^{-3}$. Thus $t_2 = 1250$ ns.

Suppose one wants to double the main memory to 64 Mbytes at the expense of reducing the disk capacity under the same budget limit. This change will not affect the cache hit ratio. But it may increase the hit ratio in the main memory, and thereby, the effective memory-access time will be reduced.

From Kai Hwuang 3e

Example - Memory Capacity Planning

You are asked to perform capacity planning for a two-level memory system.
M_1(cache) has 3 choices – 64KB, 128 KB, and 256 KB
M_2(MM) has 4 MB capacity

Given that: C_1 = 20 C_2 ; t_2 = 10 t_1 ; cache hit ratio are 0.7, 0.9 and 0.98 respectively for above three choices of M_1

1. Find t_a in terms of t_1 = 20ns
2. Find average byte cost of entire system if C_2 = \$0.2/KB
3. Compare the three memory designs

(*a*) The **average access time** is :

$$\begin{aligned} t_a &= h_1 t_1 + (1 - h_1)\, h_2 t_2 \\ &= h t_1 + (1 - h)\, 10 t_1 \\ &= (10 - 9h)\, t_1 \end{aligned}$$

If $h = 0.7$ then $t_a = 3.7\ t_1 =$ **74 ns**

If $h = 0.9$ then $t_a = 1.9\ t_1 =$ **38 ns**

If $h = 0.98$ then $t_a = 1.18\ t_1 =$ **23.6 ns**

(*b*) The **average byte cost** is :

$$C = \frac{C_1 S_1 + C_2 S_2}{S_1 + S_2}$$

$$= \frac{20 C_2 S_1 + C_2 \times 4}{S_1 + 4000}$$

$$= \frac{20 \times 0.2 S_1 + 0.2 \times 4000}{s_1 + 4000}$$

$$\boxed{C = \frac{4S_1 + 800}{S_1 + 4000}}$$

For $S_1 = 64$, the average cost is **0.26**

For $S_1 = 128$, the average cost is **0.32**

For $S_1 = 256$, the average cost is **0.43.**

(*c*) For the design choices, the product of average access time and average cost is 19.24, 12.16 and 10.15 respectively.

Example - FIFO

Consider a reference string: 7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 1, 2, 0 for a memory with 3 frames

Example - LRU

Consider a reference string: 7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2, 1, 2, 0, 1, 7, 0, 1 for a memory with 4 frames

Task2

- Recommended Readings - p157
- Group Presentation
  - Memory Capacity Planning
  - Page Replacement Algorithm
- In-Lecture Questions
  - "What new memory technologies can be used to improve hierarchical memory systems and make computational tasks faster and more efficient?“
  - "Which page replacement algorithm works best for improving memory usage and system performance?“ Explain in detail